CONCEPTS / CODE / INTERVIEWS

FastAPI Field Guide

Build understanding. Revisit the details.
Explain the decisions with confidence.

A concept-first guide with working project examples

HTTP and validation / Dependencies and SQLAlchemy

Transactions and migrations / Passwords and JWT

Testing and deployment / Architecture and tradeoffs

48 INTERVIEW QUESTIONS WITH MODEL ANSWERS

Easy / Medium / Hard / Architect

REFERENCE EDITION · NOVEMBER 2026

Contents

Read sequentially for a first pass, or jump to a concept for revision.

00 · How to use this guide 3
01 · FastAPI, ASGI, and the request lifecycle 4
02 · HTTP contracts and REST semantics 5
03 · Python types as executable API contracts 6
04 · Path, query, body, and form inputs 7
05 · Validation, serialization, and safe response models 8
06 · Dependency injection and resource lifetimes 9
07 · Synchronous code, async code, and concurrency 10
08 · Relational modeling and the ORM boundary 11
09 · Sessions, units of work, and transactions 12
10 · CRUD queries, pagination, and writable fields 14
11 · Configuration, secrets, and process state 15
12 · Password hashing and credential handling 16
13 · JWT anatomy, claims, and verification 17
14 · Login forms, bearer authentication, OAuth2, and OIDC 19
15 · Authorization and resource ownership 20
16 · Errors, middleware, and trustworthy boundaries 21
17 · Schema migrations and safe data evolution 22
18 · Tests as evidence of behavior 23
19 · OpenAPI, debugging, and reproducible checks 24
20 · Deployment, observability, and capacity 25
21 · Composition, lifecycle, and maintainable boundaries 26
22 · The wider FastAPI toolbox 27
23 · Revision sheet and design tradeoffs 28
24 · Mock interview: Easy 29
25 · Mock interview: Medium 31
26 · Mock interview: Hard 33
27 · Mock interview: Architect 35
28 · References and further study 38

00 · How to use this guide

This guide teaches concepts, then grounds them in a small working API. The business domain is deliberately ordinary: sellers register, authenticate, and manage products. That simplicity makes the important boundaries visible: a request is untrusted, a token is a claim that must be checked, a database session is temporary, and an authenticated account still needs permission to change a resource.

Read the concepts in order if FastAPI is new to you. For revision, start with a topic you cannot explain without looking at code, then answer the related interview questions aloud. An effective answer states the principle, points to an implementation decision, and describes a failure case or tradeoff.

Reading the examples

Dark panels labeled **REPOSITORY** are extracted from the actual Python files when the PDF is built. They are contextual excerpts, not independent programs: imports and surrounding definitions live in the labeled file. Panels labeled **ILLUSTRATION** show a request, command, or explanatory example. Features discussed as extensions are not claimed to exist in the API.

The code uses FastAPI, Pydantic 2, SQLAlchemy's typed ORM, Alembic, PyJWT, and pwdlib. Dependency versions are recorded under `requirements/`. The project runs synchronously with a SQLite learning database; deployment choices require additional decisions about infrastructure and identity.

The recurring example

Operation	Request and outcome
Register	POST /seller returns public account fields and stores a password hash.
Sign in	POST /login accepts a form and returns a bearer access token.
Create	POST /product assigns ownership from the authenticated account.
Browse	Authenticated sellers read a bounded, ordered product list.
Change	Only the owning seller may update or delete a product.

> The aim is to explain why the code works, what it guarantees, and where its guarantees stop. A green test suite and a JWT library do not by themselves make an internet service production-ready.

01 · FastAPI, ASGI, and the request lifecycle

An API server accepts HTTP messages, maps a request to application behavior, and sends an HTTP response. FastAPI supplies routing, dependency resolution, validation integration, and API documentation. Starlette supplies much of the underlying ASGI web machinery. Pydantic supplies data parsing and validation. Uvicorn is the server process that connects network traffic to the ASGI application.

ASGI is an interface between server and application, not a database or a programming style. It supports asynchronous request handling and long-lived connections. You can write a synchronous route inside an ASGI application; the choice of route function should match the work it performs.

ILLUSTRATION / text

```
python -m uvicorn app.main:app --reload
```

Here, `app.main` is the importable Python module and `app` is its application object. The development reload process watches files and restarts the server. Importing a file merely executes its top-level Python statements; defining `app = FastAPI()` does not itself start a network listener.



Validation and dependency failures can end the request before the route runs.

Middleware surrounds routing. FastAPI resolves dependencies and validates required inputs before calling a matching handler, although you should not rely on a particular failure order when several inputs are invalid. The handler works with validated values and returns a result. A response model validates and shapes that result before serialization. Dependency cleanup releases request-scoped resources.

The application's `create_app()` factory assembles these pieces in one place. The factory is useful for tests and configuration; it is not a requirement for every FastAPI app. Database schema changes are deliberately absent from module imports.

> Distinguish three lifetimes: server processes, reusable application resources such as connection pools, and per-request resources such as ORM sessions. Mixing them causes subtle concurrency and cleanup problems.

02 · HTTP contracts and REST semantics

HTTP methods communicate intent. GET retrieves a representation, POST commonly creates a resource, PUT replaces a resource's writable representation, PATCH describes partial changes, and DELETE removes a resource. REST conventions improve predictability, but correctness depends on documented behavior and invariants rather than whether a path is singular or plural.

The product API has a small, explicit contract. A response includes identity, ownership, and writable data. A complete PUT body includes `name`, `desc`, and `price`; omitting one is a validation error. The project does not implement PATCH.

Status	Meaning in this API
200	A read or update completed and returns a representation.
201	Registration or product creation completed.
204	Deletion completed; the response has no body.
401	Authentication is missing or invalid; bearer challenges identify the scheme.
403	Authentication succeeded but a requested write is forbidden.
404	The product does not exist.
409	Registration conflicts with an existing username or email.
422	Request data does not meet the input contract.

An operation is **idempotent** when repeating the same request has the same intended final effect. Repeating the same PUT body leaves the same product state. A second DELETE can return 404 while DELETE still has idempotent state semantics. Repeating POST can create multiple products; the application has no idempotency-key mechanism.

The deletion route in `app/api/routers/products.py` returns `Response(status_code=status.HTTP_204_NO_CONTENT)` after the owner check and commit. The caller receives no JSON representation.

The empty response matters: returning a message with a 204 response is an inconsistent contract. Likewise, 202 Accepted would suggest that work was accepted for later processing; it is inappropriate for the ordinary completed GET in this example.

> A status code is part of the public contract. Tests should assert it alongside the response body and the resulting database state.

03 · Python types as executable API contracts

Type hints make function signatures and models legible to humans and tools. Python does not generally enforce annotations by itself. FastAPI and Pydantic inspect them to build request parsers and validators, while SQLAlchemy interprets typed ORM declarations to map table columns and relationships.

`list[ProductRead]` describes a collection of response objects. `str | None` allows a null value, but does not automatically make a Pydantic field optional to omit: a default such as `= None` controls omission. `Literal["bearer"]` restricts a value to a specific string. An annotation and its default serve different purposes.

`Annotated[T, metadata]` keeps the underlying type `T` while attaching framework instructions. The application uses it both for dependency aliases and for validation constraints. The aliases avoid repeating the same dependency declaration in every route.

ILLUSTRATION / python

```python
DbSession = Annotated[Session, Depends(get_db)]
CurrentSeller = Annotated[Seller, Depends(get_current_seller)]
ProductId = Annotated[int, Path(gt=0, le=2**63 - 1)]
```

These declarations exist in `app/api/dependencies.py` and `app/api/routers/products.py`. FastAPI injects a session and authenticated seller; it parses a path segment as an integer within the supported range. A client cannot supply a trusted seller merely by naming a parameter `current_seller`.

REPOSITORY / app/api/routers/products.py / lines 55-58

```python
@router.get("/{product_id}", response_model=ProductRead)
def read_product(product_id: ProductId, db: DbSession, current_seller: CurrentSeller) -> Product:
    """Reading another seller's product is allowed once authentication succeeds."""
    return get_product_or_404(product_id, db)
```

The route's Python return annotation helps readers, editors, and static tools. Its explicit `response_model` defines the serialized API contract even though the function returns a SQLAlchemy object. The distinction is intentional.

> Use annotations to express meaning, not as a substitute for runtime validation, database constraints, or authorization. Each boundary needs the appropriate enforcement.

04 · Path, query, body, and form inputs

A request has several input channels. Path parameters identify a resource, query parameters modify a read, JSON bodies describe structured data, and forms use a different encoding. Headers carry metadata such as authorization. FastAPI uses the route declaration, type information, and explicit helpers to determine how to obtain each value.

Input	Example
Path	/product/12 supplies the product ID.
Query	?offset=20&limit=10 changes the list window.
JSON body	A ProductWrite object supplies name, description, and price.
Form body	Login supplies username and password, not a JSON object.
Header	Authorization: Bearer followed by the access token.

REPOSITORY / app/api/routers/products.py / lines 42-52

```
@router.get("", response_model=list[ProductRead])
def list_products(
    db: DbSession,
    current_seller: CurrentSeller,
    offset: Annotated[int, Query(ge=0, le=2_147_483_647)] = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
) -> list[Product]:
    """All signed-in sellers can browse; bounded pagination avoids unlimited reads."""
    # current_seller is intentionally required even though the query lists every owner.
    statement = select(Product).order_by(Product.id).offset(offset).limit(limit)
    return list(db.scalars(statement).all())
```

The list defaults to twenty rows, permits at most one hundred, and rejects negative or oversized offsets. Those are both usability choices and resource-use boundaries. Ordering by the primary key makes the result deterministic for a stable dataset.

ILLUSTRATION / json

```
{"name": "Notebook", "desc": "An A5 notebook", "price": 1299}
```

A Pydantic request model is a JSON object contract, not a raw dictionary that should be trusted. For login, `OAuth2PasswordRequestForm` declares form parsing and integrates with Swagger's authorization UI. Sending login credentials as JSON fails because the encoding does not match the route contract.

> Validate numeric ranges as well as types. An integer larger than the database driver's supported range can otherwise turn valid Python input into a server error.

05 · Validation, serialization, and safe response models

Validation answers whether data belongs in a particular contract. Serialization answers how an accepted value becomes a response representation. Pydantic models handle both, but separate request and response models make their responsibilities explicit.

REPOSITORY / app/schemas.py / lines 16-29

```
class ProductWrite(BaseModel):
    """POST and PUT accept the same complete representation (PUT is not PATCH)."""

    model_config = ConfigDict(extra="forbid")

    name: Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=100)]
    desc: Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=1000)]
    price: int = Field(
        ge=0,
        le=2_147_483_647,
        strict=True,
        description="Price in minor units: 1299 means 12.99 in the chosen currency.",
        examples=[1299],
    )
```

The product's strings are trimmed and bounded. Its price is a strict non-negative integer: a string, boolean, or float is rejected rather than silently converted. `extra="forbid"` rejects unknown fields, so a client cannot smuggle `seller_id` into creation. Some APIs intentionally allow coercion or ignore unknown fields; choose and document a policy instead of accepting defaults accidentally.

`from_attributes=True` lets a response model read named attributes from an ORM object. `SellerRead` does not define a password field. Consequently, a returned `Seller` can have a password hash internally while the public response contains only the approved attributes.

`model_dump()` produces Python data from a Pydantic model. It is appropriate for assigning the validated, explicitly writable product fields. A JSON-mode dump is useful when downstream code needs JSON-compatible values rather than Python-native objects. Secret values require deliberate extraction and should never be casually dumped into logs.

Custom validators support policies that ordinary types cannot express. The registration schema normalizes emails to lowercase, and its username constraints enforce the application's canonical identity policy. Avoid unrelated work such as database calls inside field validators; use database constraints and route logic for persistence invariants.

> Input validation and output filtering solve different problems. A validated password is still sensitive. The custom validation error handler also omits raw submitted values, since error responses can otherwise echo them.

06 · Dependency injection and resource lifetimes

Dependency injection lets a route declare what it needs without constructing everything itself. FastAPI resolves the dependency graph, including dependencies required by other dependencies. That graph can represent a database session, an authenticated account, a permission check, or a shared configuration object.

The graph here is small: routes depend on `DbSession`; protected routes also depend on `CurrentSeller`, which itself requires the session and bearer token. FastAPI normally caches a dependency's result within one request, so the authentication lookup and handler can use the same request session. This request cache is different from the process-wide `lru_cache` used for settings.

REPOSITORY / app/database.py / lines 50-54

```
def get_db() -> Generator[Session, None, None]:
    # A context manager always closes the session and rolls back uncommitted work.
    # Routes explicitly commit writes, so a failed request cannot silently persist.
    with SessionLocal() as session:
        yield session
```

A dependency that yields can pair acquisition with cleanup. The session context manager closes the session and rolls back uncommitted work when the dependency scope ends, including failure paths. Commits remain explicit in routes. Never depend on a cleanup phase to commit a write after the response's success has already been determined.

Directly calling `get_db()` produces a Python generator; it does not perform FastAPI's injection. Similarly, `Depends(...)` has meaning when FastAPI interprets the callable, not as magic in an arbitrary helper function. Ordinary helpers receive ordinary values.

In tests, `application.dependency_overrides[get_db]` replaces the session provider while keeping real route behavior and authentication. This is a practical benefit of explicit dependency boundaries.

> An ORM session is mutable transaction state. Create one per request; do not keep one global session for all clients or reuse it across unrelated concurrent tasks.

07 · Synchronous code, async code, and concurrency

Concurrency is the ability to make progress on multiple tasks during overlapping time periods. Parallelism is simultaneous execution. Async I/O is useful when an operation can suspend while waiting for a network or database response. It does not make CPU work cheaper, make blocking libraries asynchronous, or remove database contention.

FastAPI executes ordinary `def` handlers and synchronous dependencies through a thread pool. An `async def` handler runs on the event loop and should await non-blocking operations. A synchronous database call made directly inside an async handler can block the event loop, affecting unrelated requests.

REPOSITORY / app/api/routers/products.py / lines 31-39

```
@router.post("", response_model=ProductRead, status_code=status.HTTP_201_CREATED)
def create_product(request: ProductWrite, db: DbSession, current_seller: CurrentSeller) ->
Product:
    # Ownership comes from the verified token, never a client-supplied seller_id.
    product = Product(**request.model_dump(), seller_id=current_seller.id)
    db.add(product)
    # Commit before returning success; refresh loads database-generated values.
    db.commit()
    db.refresh(product)
    return product
```

This handler uses a synchronous SQLAlchemy `Session`. Keeping it as `def` makes the execution model clear. Password verification is also synchronous and computationally expensive. Wrapping that work in an async function without offloading it would not improve throughput.

Work	Sensible starting point
Blocking synchronous ORM	A synchronous handler and request session.
Async database or HTTP client	Async handler with awaitable operations and an appropriate client/session.
Heavy CPU computation	A bounded process/worker system, with deliberate capacity limits.
Slow durable side effect	A persistent job queue with retries and idempotency.

An async conversion is a whole-path change: driver, engine, session type, dependency, queries, and tests must agree. It should follow measurement or a concrete concurrency need. Even with async I/O, worker limits, connection pools, database locks, and memory remain real constraints.

> A helper called normally inside a handler runs normally. FastAPI's thread-pool handling does not automatically offload every synchronous function you happen to call.

08 · Relational modeling and the ORM boundary

A relational model describes persistent entities and their invariants. A primary key identifies a row. A foreign key connects rows across tables. A unique constraint prevents conflicting identities. A check constraint rejects invalid stored values. These constraints protect every writer, including code that bypasses HTTP validation.

SQLAlchemy's ORM maps Python attributes to relational columns. `Mapped[T]` describes mapped data, and `mapped_column()` declares column details. Nullability can be inferred from the annotation: these required fields are not nullable. A relationship exposes related objects; the underlying foreign key enforces the connection in the database.

REPOSITORY / app/models.py / lines 26-43

```
class Product(Base):
    """A product always has an owner and a price in integer minor currency units."""

    __tablename__ = "products"
    __table_args__ = (
        CheckConstraint("price >= 0 AND price <= 2147483647", name="ck_products_price"),
    )

    # Non-optional Mapped types make these columns NOT NULL in the database schema.
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    desc: Mapped[str] = mapped_column(String(1000))
    # Integer minor units avoid floating-point rounding errors for money.
    price: Mapped[int] = mapped_column(Integer)
    seller_id: Mapped[int] = mapped_column(
        ForeignKey("sellers.id", ondelete="RESTRICT"), index=True
    )
    seller: Mapped[Seller] = relationship(back_populates="products")
```

Each product belongs to a seller. The foreign key uses RESTRICT semantics, and SQLite foreign-key checks are explicitly enabled when a connection opens. A Python relationship alone would not prevent orphaned rows in a database with enforcement disabled.

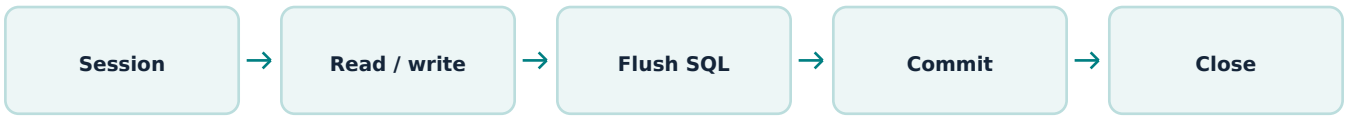
The seller table has unique username and email constraints. Registration normalizes identities before inserting. The database then protects uniqueness even if two requests race. Application-level existence checks alone cannot guarantee that another request does not insert between a check and a write.

Money is represented in integer minor units to avoid binary floating-point rounding. That does not solve taxes, currencies with different minor-unit scales, exchange rates, or accounting rules; those would require a richer domain model. No unnecessary commerce model is introduced here.

> Pydantic models describe messages. ORM models describe persistence. They can share concepts while remaining separate, allowing each contract to evolve deliberately.

09 · Sessions, units of work, and transactions

A database engine manages connectivity and pooling. A session tracks ORM objects and transaction state for a unit of work. A transaction makes a set of database changes succeed or fail together according to the database's guarantees. These are related abstractions, not interchangeable names for a connection.



If a write fails: rollback. Session cleanup still runs after the request.

Operation	Meaning
add	Enlist a new ORM object in the session.
flush	Send pending SQL within the current transaction; it is not durable yet.
commit	Flush pending changes and commit the transaction.
refresh	Reload an object's values from the database.
rollback	Abandon the current transaction and recover failed transaction state.
close	Release session resources; uncommitted work is not committed for you.

REPOSITORY / app/api/routers/sellers.py / lines 14-34

```
@router.post("", response_model=SellerRead, status_code=status.HTTP_201_CREATED)
def create_seller(request: SellerCreate, db: DbSession) -> Seller:
    """Hash passwords once; store only the hash, and never return it."""
    seller = Seller(
        username=request.username,
        email=str(request.email),
        hashed_password=password_hash.hash(request.password.get_secret_value()),
    )
    db.add(seller)
    try:
        db.commit()
    except IntegrityError:
        # A database constraint handles simultaneous registrations safely too.
        db.rollback()
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Username or email already registered",
        ) from None
    db.refresh(seller)
    # SellerRead filters this ORM instance so the stored password hash cannot leak.
    return seller
```

Failures and concurrency

The route catches the expected uniqueness conflict, rolls back, and returns a useful 409 response without exposing SQL details. A session that encountered a failed flush or commit must be rolled back before it is reused. The request dependency still closes the session afterward.

Isolation controls what concurrent transactions observe. A transaction boundary does not automatically prevent lost updates in a read-modify-write flow. Two owners' requests can both load an old value and overwrite each other. This simple API has no optimistic version column; a more demanding application should define its concurrency contract explicitly.

External side effects are outside the database transaction. Sending an email and then committing cannot be rolled back as one atomic unit. For reliable cross-system workflows, consider an outbox and a worker rather than pretending one ORM transaction controls every effect.

> Successful flush is not successful commit. A test that reuses one session can accidentally see uncommitted changes; the HTTP fixtures deliberately create separate request sessions.

10 · CRUD queries, pagination, and writable fields

CRUD is simple when each operation has a clear input contract, authorization rule, transaction boundary, and output contract. Creation accepts only writable fields; identity and ownership come from trusted server context. Reads handle missing rows consistently. Updates change an allowed set of attributes. Deletes verify existence and permission before removing data.

REPOSITORY / app/api/routers/products.py / lines 61-76

```
@router.put("/{product_id}", response_model=ProductRead)
def update_product(
    product_id: ProductId,
    request: ProductWrite,
    db: DbSession,
    current_seller: CurrentSeller,
) -> Product:
    """PUT requires all writable fields; it never transfers ownership."""
    product = get_product_or_404(product_id, db)
    require_owner(product, current_seller.id)
    # Only schema-approved fields can be assigned; IDs and ownership are not writable.
    for field, value in request.model_dump().items():
        setattr(product, field, value)
    db.commit()
    db.refresh(product)
    return product
```

The request model limits the fields used by `model_dump()`, so the loop does not mass-assign arbitrary request keys. This would be unsafe if it iterated an unrestricted dictionary containing fields such as `seller_id`, `id`, or a future administrative flag.

`db.get(Product, id)` performs a primary-key lookup. `select(Product)` builds a composable SQL statement. `db.scalars(statement)` yields ORM objects from the selected scalar column. SQLAlchemy binds values separately from SQL text; avoid interpolating request values into SQL strings.

Offset pagination is easy to explain and sufficient for this bounded example. It can become expensive at large offsets and can shift when rows are inserted or deleted between requests. A cursor based on a stable, unique ordering is often more suitable for large or frequently changing datasets.

Relationships can also create performance surprises. Accessing a lazily loaded relationship once per result can produce an N+1 query pattern. Measure emitted queries and use an appropriate eager-loading strategy when returning related data. The current response contract avoids serializing relationship graphs accidentally.

> Add a service abstraction when business operations are reused or require coordinated rules. A layer that merely forwards each CRUD call does not by itself improve this small application's correctness.

11 · Configuration, secrets, and process state

Configuration is deployment-specific data: database location, allowed hosts, token lifetime, issuer, audience, and signing secret. Keeping it separate from business logic lets the same application run in development, tests, and deployment without editing source code.

Pydantic Settings loads environment values into typed fields and can read a dotenv file. Environment variables take precedence over the file. Missing or invalid required configuration should stop startup rather than leave an application running with an accidental public key or broken database target.

REPOSITORY / app/core/config.py / lines 48-51

```
@lru_cache
def get_settings() -> Settings:
    """Parse configuration once per process; missing secrets stop startup early."""
    return Settings()
```

The settings object is cached once per Python process. Changing the environment after startup does not transparently replace cached settings; restart the process or explicitly clear the cache in a controlled test. Multiple workers each have a cache and must receive consistent configuration.

`SecretStr` masks a value in common representations. It is not encryption and does not prevent a developer from calling `get_secret_value()`. Extract the signing key only at the cryptographic boundary. Do not log settings objects, request credentials, or connection strings that may contain passwords.

The example dotenv file contains a rejected placeholder. The real `.env` is ignored by Git. The signing key should be generated randomly; a length check alone cannot prove entropy. Deployment should supply secrets through its secret-management system and define a rotation procedure.

> A cached settings object is acceptable process configuration. A cached ORM session is shared mutable request state. Treating those as the same caching problem leads to unsafe designs.

12 · Password hashing and credential handling

Password storage requires a slow, salted password-hashing algorithm. Hashing is one-way verification; encryption is reversible with a key. A fast general-purpose hash such as SHA-256 is unsuitable by itself for password storage because attackers can test guesses cheaply.

New registrations use Argon2id through pwdlib. The encoded hash includes algorithm parameters and salt information, so the application can store one hash string and verify it later. The plaintext password is used for the calculation, then excluded from persisted and public representations.

ILLUSTRATION / python

```
password_hash = PasswordHash.recommended()
```

This declaration is in `app/core/security.py`. The recommended configuration uses Argon2id. Successful verification can also return a replacement hash when stored cost parameters need upgrading; login saves that replacement only after confirming the password. This keeps existing accounts current as the hashing policy changes.

Registration accepts 12–128 characters without trimming spaces. There are no arbitrary character-composition rules. Password policy and hashing cost are separate decisions: a long passphrase still needs an expensive salted hash. The application uses one algorithm, Argon2id, to keep the verification policy small and explicit.

Login returns the same 401 message for an unknown username and a wrong password. A dummy hash is verified for an unknown account to reduce one timing discrepancy. This does not make the complete endpoint perfectly constant-time or replace shared rate limits and abuse monitoring.

> Hashing cost consumes CPU and memory by design. Limit login traffic before it becomes expensive work, and benchmark hashing settings against both legitimate latency and attack resistance.

13 · JWT anatomy, claims, and verification

A JWT typically has a header, a payload, and a signature. Its encoded payload is readable; a signature does not encrypt it. A bearer token grants access to whoever possesses it, so avoid putting sensitive data in the claims and protect the token in transit and storage.

REPOSITORY / app/core/security.py / lines 18-32

```
def create_access_token(seller_id: int) -> str:
    """Sign short-lived identity claims; a JWT payload is readable, not encrypted."""
    settings = get_settings()
    now = datetime.now(UTC)  # Aware UTC timestamps avoid local-time ambiguity.
    return jwt.encode(
        {
            "sub": str(seller_id),  # A stable ID survives a future username change.
            "iat": now,
            "exp": now + timedelta(minutes=settings.access_token_expire_minutes),
            "iss": settings.jwt_issuer,
            "aud": settings.jwt_audience,
        },
        settings.jwt_secret_key.get_secret_value(),
        algorithm=ALGORITHM,
    )
```

Claim	Meaning and check
sub	Subject: the stable seller ID encoded as a string.
iat	Issued-at time; the verifier checks that it is not unacceptably in the future.
exp	Expiration: after this instant, the token must be rejected.
iss	Issuer: identifies the service that issued the token.
aud	Audience: identifies the intended recipient.

The application fixes HS256 and validates tokens using its own configuration. A verifier must not select its trusted algorithm merely because the untrusted header names one. It also requires the expected claims; validating an optional expiration only when it happens to exist would leave a missing-expiration hole.

REPOSITORY / app/core/security.py / lines 35-57

```
def decode_access_token(token: str) -> int:
    """Validate the signature AND required claims; return only the seller ID."""
    settings = get_settings()
    payload = jwt.decode(
        token,
        settings.jwt_secret_key.get_secret_value(),
        algorithms=[ALGORITHM],
        issuer=settings.jwt_issuer,
        audience=settings.jwt_audience,
        options={"require": ["sub", "iat", "exp", "iss", "aud"]},
    )
    # Restrict the signed subject to IDs the database can represent safely.
    # Signature/claim validation happens first; account existence is checked by the API.
    subject = payload["sub"]
    if (
        not isinstance(subject, str)
        or not subject.isascii()
        or not subject.isdecimal()
        or len(subject) > 19
        or not 0 < int(subject) <= 2**63 - 1
    ):
        raise jwt.InvalidTokenError("Invalid seller ID")
    return int(subject)
```

Cryptographic validity is only one stage. The subject must be a valid supported seller ID, and a protected request then looks up that account in the database. A signed token for a missing account is rejected.

HS256 uses the same secret to sign and verify. This is compact for one application, but every service with that secret can issue tokens. A distributed identity design may prefer asymmetric signatures, public verification keys, key IDs, and managed rotation. Those additions should follow a real trust boundary.

> Short token lifetimes reduce the window for stolen-token use; they do not provide immediate logout or per-token revocation. This project intentionally has no refresh or revocation subsystem.

14 · Login forms, bearer authentication, OAuth2, and OIDC

Authentication establishes who a caller is. The local login endpoint verifies a seller's username and password, then issues a short-lived token. Later requests present that token in the Authorization header instead of resending the password.

ILLUSTRATION / text

```
Authorization: Bearer <access_token>
Content-Type: application/x-www-form-urlencoded
```

The first header belongs on a protected resource request. The content type describes the login form request, not every API request. Product creation still uses JSON. FastAPI's OAuth2 helper classes make these contracts visible to OpenAPI and Swagger UI.



Later requests send the bearer JWT; ownership is checked separately for writes.

`OAuth2PasswordBearer(tokenUrl="login")` extracts a bearer credential and describes the token endpoint. It does not validate passwords, verify JWT signatures, check database accounts, or enforce ownership. Those are explicit parts of the application.

OAuth2 is a delegated authorization framework; OIDC adds standardized identity information on top of OAuth2. A simple local username/password flow is not a complete external identity provider. It also does not cover consent, identity federation, multi-factor authentication, account recovery, or session management.

The password-form flow is retained as a teaching example. For delegated login or SSO, use an identity provider and Authorization Code with PKCE rather than expanding this example into a home-grown authorization service. RFC 9700 rejects the resource owner password credentials grant because it exposes user credentials to the client and does not fit modern authentication requirements.

The token response sets no-store caching headers. A browser client still needs an explicit storage strategy: script-readable storage faces token theft through XSS, while cookie-based authentication brings cookie configuration and CSRF considerations. There is no universal safe choice independent of the application threat model.

> A successful Swagger Authorize action proves the configured flow can obtain a token. It does not prove that every protected route validates it or checks permissions; that requires tests.

15 · Authorization and resource ownership

Authorization determines what an authenticated account may do. It should be checked at the resource and operation boundary. Authentication alone does not imply permission to edit another seller's product.

REPOSITORY / app/api/dependencies.py / lines 20-36

```
def get_current_seller(token: Annotated[str, Depends(oauth2_scheme)], db: DbSession) -> Seller:
    """Resolve a trusted account or raise 401; ownership is a separate route rule."""
    error = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or expired credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        seller_id = decode_access_token(token)
    except InvalidTokenError:
        raise error from None

    # A valid signature is not enough: the account must still exist.
    seller = db.get(Seller, seller_id)
    if seller is None:
        raise error
    return seller
```

This dependency authenticates the caller by checking token claims and resolving a real account. It returns a Seller on success; a failure raises an HTTP exception before the protected handler runs.

REPOSITORY / app/api/routers/products.py / lines 25-28

```
def require_owner(product: Product, seller_id: int) -> None:
    """Authentication answers who; authorization answers what they may change."""
    if product.seller_id != seller_id:
        raise HTTPException(status_code=403, detail="Only the owner can modify this product")
```

All signed-in sellers can browse products, so a forbidden write returns 403 without concealing a resource that is already readable. In an API where even existence is confidential, a consistent 404 policy may be appropriate. Choose the disclosure policy from the domain, not as a universal status-code rule.

Creation derives `seller_id` from the authenticated account. Accepting that field from the request would allow identity spoofing even if later edits checked ownership correctly. Response schemas and input schemas support authorization by limiting which fields can cross the boundary, but they do not replace the owner check.

For more complex domains, permission rules may involve roles, scopes, organizations, memberships, resource states, or explicit grants. Apply tenant boundaries to both reads and writes; do not rely on a client-provided filter to isolate tenants. The simple project deliberately implements ownership rather than introducing an unused role framework.

> Test authorization with at least two accounts and verify state after a denied write. A 403 response alone does not prove the implementation avoided changing the database first.

16 · Errors, middleware, and trustworthy boundaries

Expected failures belong in the API contract. An invalid token is not an internal server crash, and a missing product is not a successful response with an empty message. Raise an HTTP exception at a point that still allows cleanup, and return a concise error that a client can act on.

REPOSITORY / app/api/routers/products.py / lines 17-22

```
def get_product_or_404(product_id: int, db: Session) -> Product:
    """Share the lookup between read/update/delete to keep 404 behavior consistent."""
    product = db.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="Product not found")
    return product
```

Unexpected errors require a different boundary. The application returns a generic database failure response and logs the internal exception. SQL bound parameters are hidden by the engine configuration. That reduces accidental data disclosure; it does not authorize logging passwords, tokens, raw bodies, or database URLs elsewhere.

REPOSITORY / app/main.py / lines 35-41

```
@application.exception_handler(RequestValidationError)
async def validation_error_handler(
    request: Request, exc: RequestValidationError
) -> JSONResponse:
    # FastAPI normally echoes invalid inputs. Avoid returning passwords or tokens.
    errors = [{key: error[key] for key in ("loc", "msg", "type")} for error in exc.errors()]
    return JSONResponse(status_code=422, content={"detail": jsonable_encoder(errors)})
```

Removing the raw `input` and context fields prevents the usual validation response from echoing a submitted password. This is an example of considering failure-path data exposure, not just shaping successful responses.

Middleware operates around requests and responses. TrustedHostMiddleware rejects unexpected Host headers. CORS governs which browser origins may read cross-origin responses; it is not authentication, and non-browser clients are not stopped by it. No frontend is part of this project, so CORS is not enabled speculatively. Add explicit origins if an actual browser client needs them.

HTTPS protects network traffic. A Host allowlist does not provide TLS, rate limiting, or a request-size limit. Those controls are deployment responsibilities here. If cookies are used for authentication in an extension, evaluate CSRF protection separately from CORS.

The health endpoint queries the database as a connectivity check. It does not validate every table, prove migrations are current, or test external dependencies. A production design may separate lightweight liveness from dependency-aware readiness to avoid restart loops when a dependency is temporarily unavailable.

> Never convert every exception into a successful 200 response. Useful status codes and sanitized messages help clients distinguish retryable conditions from invalid requests.

17 · Schema migrations and safe data evolution

A model definition describes desired structure; a migration records a versioned transition. Creating tables on application import does not update existing columns, handle data transformations, or coordinate concurrent deployments. Alembic gives schema changes an explicit release path.

ILLUSTRATION / text

```
python -m alembic upgrade head
python -m alembic revision --autogenerate -m "add a reviewed change"
python -m alembic check
```

`upgrade head` applies pending revisions to the configured database. Autogeneration compares model metadata with the current database and proposes changes. `check` detects whether another autogenerated revision would contain operations; it is useful for catching drift. None of these eliminates the need to review a migration.

The initial migration creates the constrained seller and product tables. Tests apply it to an empty temporary file, compare it with ORM metadata, apply it again, downgrade, and upgrade once more. These tests exercise the actual migration path rather than substituting table creation for release behavior.

The app uses `data/app.db`, created through the initial migration on a fresh installation. Pointing the app at an existing database does not migrate it, and blindly stamping a revision only changes bookkeeping; it does not make an incompatible schema correct. Review the actual schema and plan explicit data transformations when importing outside data.

For a live system, prefer compatible changes. Add a nullable field or compatible table, deploy code that understands both states, backfill safely, then tighten constraints or remove old fields in a later release. A rename that looks simple in Python can break old workers during a rolling deployment.

> A downgrade is code that may destroy data, not a replacement for a backup. Design recovery around the actual database, schema change, and release strategy.

18 · Tests as evidence of behavior

A useful test checks a meaningful contract or invariant. Unit tests isolate a small rule. Integration tests exercise cooperating components. HTTP tests verify routing, validation, authentication, response filtering, and persistence together. End-to-end deployment tests additionally include real network, process, and infrastructure behavior.

The HTTP fixture creates a fresh app and replaces only the database dependency. The authentication code, ORM models, and routes remain real. Separate request sessions catch missing commits that a shared test session could hide. StaticPool lets the in-memory SQLite database be used from TestClient's worker threads.

REPOSITORY / tests/test_products.py / lines 49-57

```
def test_other_seller_can_read_but_cannot_modify(client, owner, account_factory):
    _, owner_headers = owner
    _, other_headers = account_factory("bob")
    product = client.post("/product", json=PRODUCT, headers=owner_headers).json()
    path = f"/product/{product['id']}"
    assert client.get(path, headers=other_headers).status_code == 200
    assert client.put(path, json={**PRODUCT, "price": 1}, headers=other_headers).status_code ==
403
    assert client.delete(path, headers=other_headers).status_code == 403
    assert client.get(path, headers=owner_headers).json() == product
```

This test checks both the forbidden result and the unchanged state afterward. JWT tests cover missing claims, bad signatures, wrong algorithms, expired tokens, incorrect issuer/audience, malformed subjects, and deleted accounts. Registration tests check identity normalization, duplicate conflicts, hashes at rest, and response filtering.

Parametrization expresses several examples of one rule without duplicating test functions. Fixtures describe setup and cleanup boundaries. Test overrides must be cleared so one test cannot silently change the next test's environment.

Coverage reports which statements ran, not whether all important behaviors were asserted. The refactored application reached 100% statement coverage in local verification, but that does not demonstrate complete concurrency, performance, infrastructure, or security coverage. If deployment moves to PostgreSQL, test against PostgreSQL as well; SQLite does not reproduce every backend behavior.

> A useful regression test fails for the original defect, passes for the corrected behavior, and avoids depending on incidental implementation details.

19 · OpenAPI, debugging, and reproducible checks

OpenAPI describes the public API contract: paths, methods, schemas, parameter locations, status codes, and authentication requirements. Swagger UI and ReDoc render that description. They are excellent exploration tools, but the generated contract should be inspected and tested rather than assumed to match an undocumented intention.

The project declares response models and status codes explicitly. Swagger understands the login endpoint through the bearer helper's token URL. Interactive docs help a learner complete the registration, login, and CRUD sequence without writing a separate client first.

For debugging, the editor must attach its debugger to the process executing the handler. Running an ordinary terminal server does not automatically connect VS Code's breakpoints. The launch configuration starts the Uvicorn module under debugpy using the project's virtual environment.

ILLUSTRATION / text

```
Run and Debug > Debug FastAPI > F5
Open /docs and execute the endpoint containing the breakpoint.
```

A route breakpoint runs when that route is called, not when its Python function is merely defined. A single-process debug profile avoids reload-process confusion. Watch the parsed request, current seller ID, ORM object, and exception path; do not display real credentials during a screen recording.

ILLUSTRATION / text

```
python -m ruff check .
python -m ruff format --check .
python -m pytest --cov --cov-report=term-missing
```

CI repeats these checks after pushes and pull requests. It is a consistency mechanism, not deployment approval. Dependency inputs are separated from compiled pins so reviewers can distinguish intentional package choices from the resolved graph. Update pins deliberately, review relevant advisories, and verify a clean installation.

> A reproducible local setup, a readable README, and real failure-case tests are part of an educational project's quality. They let another developer verify the claims independently.

20 · Deployment, observability, and capacity

Production readiness is a property of the application together with its operating environment. It includes restart behavior, secure transport, secret distribution, data durability, resource limits, monitoring, and a release procedure. This repository implements a small, inspectable application and documents what the deployment must supply.

Concern	Responsibility in this design
HTTPS and ingress controls	Reverse proxy or hosting platform.
Shared rate and size limits	Gateway/proxy before expensive login work.
Schema changes	One explicit Alembic release step, with backups.
Durable storage	Persistent database with a recovery procedure.
Process recovery	A supervisor/platform restarts failed server workers.
Visibility	Metrics, sanitized logs, alerts, and useful health probes.

Do not use development reload as a process supervisor. If multiple workers are used, each has its own memory, settings cache, and connection pool. They need the same signing configuration. A Python dictionary cannot be a shared rate limiter or a reliable revocation registry across workers.

SQLite keeps local learning simple, but concurrent writes and multi-replica deployment change the requirements. A managed PostgreSQL database is a reasonable next step when contention, operational recovery, or independent replicas matter. Changing the URL also requires the correct driver, migrations, connection-pool sizing, and backend-specific tests.

Useful signals include request rate, error rate, latency percentiles, database pool wait time, query duration, and authentication failure trends. A fast average can hide a poor tail latency. Request identifiers help connect events without logging credentials.

Capacity planning should include the total database connections across all workers and replicas, password-hashing cost, and gateway limits. Add indexes based on actual query patterns and measured plans. Caching a public product representation is different from caching an authorization decision; the latter needs especially careful invalidation.

A cache needs a key, an access scope, and invalidation rules. Never place a private response under a public key. ETags and conditional requests can avoid retransmitting unchanged data; an enforced If-Match/version precondition can also support optimistic updates. These are possible extensions, not behaviors currently implemented by the API.

> Start with measured bottlenecks. More workers, async syntax, and caching can each increase complexity or pressure on a dependency if applied without a model of the workload.

21 · Composition, lifecycle, and maintainable boundaries

An APIRouter groups operations with shared prefixes, tags, dependencies, or responses. Including a router registers its operations on an application. A router is not a separate server, and splitting a file does not automatically create an architectural boundary. Choose modules that separate responsibilities a reader can explain.

The project groups seller registration, login, and product operations. Shared token and password functions live outside routers so authentication logic can be reused without importing an HTTP handler. Request schemas stay separate from persistent models. Ordinary lookup and ownership helpers remain close to the small CRUD flow they support.

A factory such as `create_app()` centralizes assembly. An application lifespan context is appropriate when a process needs explicit startup and shutdown management for an async HTTP client, a loaded model, or another shared resource. The ORM engine is a reusable pool; ORM sessions are still created per request. Do not put schema migrations or slow, uncontrolled work into every worker's startup.

If a future operation coordinates several entities, a service function can make the business transaction explicit. A repository abstraction can help when query behavior is reused or persistence choices need isolation. These layers should have a purpose beyond moving the same lines into more files.

Documentation and examples also need stable boundaries. The PDF builder extracts code from the repository, while the prose explains the concept. It does not maintain a second copy of the application that can silently diverge.

> Standardization should reduce the number of decisions needed to understand the project. It should not require a newcomer to follow five forwarding layers to find one database write.

22 · The wider FastAPI toolbox

The CRUD/JWT application is a foundation, not a demonstration of every FastAPI feature. The following concepts are useful extensions; they are discussed here without adding unrelated endpoints to the codebase.

Richer data contracts

Pydantic supports nested models, collections, enums, UUIDs, datetimes, and field/model validators. Use a nested object when it represents a meaningful substructure, not just to make JSON appear sophisticated. Define timezone and serialization policies for datetimes. A value that can be null and a field that can be omitted are different contracts.

For partial updates, a dedicated PATCH schema can make fields omittable and `model_dump(exclude_unset=True)` can identify the supplied values. Carefully distinguish an omitted field from an explicit null. The current PUT intentionally requires a complete body and does not pretend to implement those PATCH semantics.

Files, streams, and large responses

File and form inputs use multipart/form-data when combined. UploadFile provides a file-like interface appropriate for uploads; a declared byte value loads content directly. Validate type and size at trustworthy boundaries, use generated storage names, and never trust the submitted filename as a filesystem path. A JSON body and multipart parsing are different request encodings.

StreamingResponse can send chunks rather than constructing an entire response in memory. FileResponse serves file content with appropriate behavior. Streaming changes failure semantics: an error after headers are sent cannot be reported as though no response started. Resources must remain valid for the stream's actual lifetime.

Background work

FastAPI background tasks are convenient for modest, non-critical work associated with a response. They run in the application process, not in a durable job system. A crash or redeployment can lose work. Durable email, reports, or payment workflows need persisted jobs, retries, idempotency, and a strategy for poison messages. Pass identifiers to workers and open a fresh session there rather than reusing a request session that may be closed.

WebSockets and server-initiated updates

WebSockets maintain a bidirectional connection and require connection-level authentication, message validation, cleanup, and backpressure policies. A JWT supplied once at connection establishment raises a separate question about expiry during a long-lived connection. Multiple workers also need a shared messaging mechanism when updates should reach clients connected elsewhere.

Server-sent events are useful for one-way server-to-client streams. Choose transport from the communication pattern; ordinary HTTP requests remain sufficient for this product API.

23 · Revision sheet and design tradeoffs

Explain these distinctions without opening the code

Distinction	One-sentence reminder
FastAPI / Uvicorn	Framework application behavior versus the ASGI server process.
Schema / ORM model	Message contract versus persistent table mapping.
Validation / authorization	Acceptable input versus permitted action.
Authentication / authorization	Verified identity versus allowed operation.
Hashing / encryption	One-way password verification versus reversible protection.
JWT / session lifecycle	Signed claims versus the complete login, renewal, and revocation policy.
Engine / session	Reusable connectivity pool versus per-unit-of-work state.
Flush / commit	SQL sent in a transaction versus a committed transaction.
PUT / PATCH	Complete writable representation versus explicitly partial changes.
401 / 403	Authentication required or invalid versus authenticated but forbidden.
Sync / async	Blocking execution model versus cooperative awaitable work.
Migration / create_all	Versioned evolution versus initial missing-table creation.
Liveness / readiness	Process responsiveness versus ability to serve useful work.

Deliberate project choices

- Product routes require authentication; only owners can mutate products.
- Synchronous handlers match the ORM; integer minor units simplify money representation.
- Short-lived access tokens keep identity mechanics inspectable; renewal and revocation remain explicit extensions.
- Migrations create a fresh constrained database; extra business layers wait for actual reuse or rules.

A useful revision exercise

Create two sellers. Create a product under the second account and explain why its owner is not hardcoded to ID 1. Try editing it using the first account. Then inspect a token's payload without trusting it, explain every required claim, and identify the function that turns it into an actual account. Finally, explain what happens to the transaction and response when a duplicate registration fails.

24 · Mock interview: Easy

Use these questions to check foundational vocabulary. Give the principle first, then relate it to one concrete behavior in the project.

E01 · What does FastAPI provide, and what does Uvicorn do?

Model answer. FastAPI defines the application: routing, input validation integration, dependencies, and OpenAPI contracts. Uvicorn runs the ASGI server that receives network requests and calls that application. Pydantic handles data validation, and Starlette provides underlying web machinery. In this project, `app/main.py` exposes `app`, and Uvicorn runs it using `app.main:app`.

E02 · What does app.main:app mean in the run command?

Model answer. The part before the colon is an importable Python module, and the part after it names the application object inside that module. It is not a filesystem path with an arbitrary variable name. The working directory and import path must make `app.main` importable. The project runs from its root so both imports and the documented relative database URL behave predictably.

E03 · Why use APIRouter?

Model answer. A router groups related operations and can share a prefix, tags, dependencies, and response metadata. Including it registers those operations with the application. It helps organize the product, seller, and login concerns without creating separate servers. It also does not replace an explicit business or security boundary; the route still needs the correct dependencies and owner checks.

E04 · How do path, query, and body parameters differ?

Model answer. A path parameter identifies a resource, such as the product ID. Query parameters modify the request, such as the pagination limit. A body contains structured input, such as the fields for a new product. The route signature and Pydantic model describe these channels. Login uses form data, which is a separate body encoding from the JSON product representation.

E05 · Why are API schemas separate from database models?

Model answer. A schema describes an accepted or returned message, while an ORM model describes persistent columns and relationships. They often contain different fields and constraints. The seller table needs a password hash, but SellerRead must not expose it. Separating them also allows database structure and public API contracts to change with more control.

E06 · How does response_model help protect data?

Model answer. It defines and validates the public response shape. A route can return a Seller ORM object, while SellerRead includes only ID, username, and email. The password hash is excluded because it is not part of the response contract. This is one boundary, not permission checking: an approved field may still be private if the caller is not authorized to read that resource.

E07 · What is dependency injection in this application?

Model answer. A handler declares a session or authenticated seller in its signature, and FastAPI resolves the provider functions. DbSession yields a request-scoped session. CurrentSeller extracts a bearer token, verifies it, and loads the account using that session. Tests can replace the session provider without rewriting the route or bypassing the authentication logic.

E08 · Which success status codes does CRUD use here?

Model answer. Creation returns 201, reads and completed updates return 200, and deletion returns 204 without a body. A missing product returns 404. A full PUT update returns the updated product rather than a set or an ambiguous message. These choices are part of the documented contract and are asserted in the HTTP tests.

E09 · What is the difference between 401 and 403?

Model answer. A 401 means valid authentication is required or was not supplied; bearer authentication failures also include a WWW-Authenticate header. A 403 means the authenticated caller is forbidden from the operation. In this project, an expired token gets 401, while another seller's attempt to delete an existing readable product gets 403.

E10 · Is a JWT payload secret?

Model answer. No. The payload is encoded and can be read without the signing secret. The signature provides integrity and authenticity when properly verified; it does not encrypt the claims. The project puts a seller ID and standard token claims in the payload. Passwords and sensitive profile data do not belong in it.

E11 · Why hash a password rather than store or encrypt it?

Model answer. Authentication needs to check a submitted password, not recover the original. A slow salted password hash supports verification while making stolen hashes harder to attack. Reversible encryption introduces a recovery key that can expose every password if compromised. The application uses Argon2id for new passwords and never includes hashes in seller responses.

E12 · Why does a route breakpoint not stop immediately after startup?

Model answer. Defining a function registers its handler but does not execute its body. The debugger pauses there only when a matching request reaches the function. Start the included Debug FastAPI profile with F5, then execute the endpoint through Swagger or another client. The debugger must be connected to the actual server process; a separate ordinary terminal process is independent.

25 · Mock interview: Medium

These questions connect framework features with correctness. Explain both the normal path and one relevant failure path.

M01 · Why does JSON login fail when product creation uses JSON successfully?

Model answer. Login declares OAuth2PasswordRequestForm, so it expects username and password as form fields. Product creation declares a Pydantic body model, so it expects the JSON representation. Encoding is part of the endpoint contract. I would inspect the Content-Type and request payload rather than assume a correct dictionary is sufficient. Swagger's authorization UI submits the login form automatically.

M02 · Why must the current-seller dependency return a database account?

Model answer. A handler needs a trusted identity to make ownership decisions. Merely decoding claims and returning nothing leaves the dependency's consumer with no authenticated seller. The dependency also checks that the subject still resolves to an existing account; a signed token is not proof that the account still exists. The function returns Seller or raises a consistent 401.

M03 · Why reject seller_id in a product creation request?

Model answer. Ownership is trusted authorization state, so the server derives it from the verified seller. Accepting an arbitrary seller_id would let a caller create products on behalf of another account. ProductWrite rejects unknown fields, and create_product explicitly supplies `current_seller.id`. Both the input contract and the assignment are tested, including an account whose ID is not 1.

M04 · Why is a pre-insert duplicate check insufficient?

Model answer. Two concurrent requests can both see no existing row and then both insert. A database unique constraint arbitrates that race. Registration catches the resulting IntegrityError, rolls the session back, and returns 409. A pre-check can improve a message, but it is not the final guarantee. The input normalization policy must also be consistent before values reach the constraint.

M05 · Explain flush, commit, refresh, and rollback.

Model answer. Flush sends pending SQL inside the current transaction and can produce database-generated values; it does not commit. Commit makes the transaction durable according to the database's configuration. Refresh reloads the object's state. Rollback abandons the transaction and recovers from a failed transaction. The create routes commit and refresh before returning; registration explicitly rolls back an expected uniqueness conflict.

M06 · How does a yield dependency help with database sessions?

Model answer. It pairs session acquisition with cleanup for the dependency's scope. The context manager closes the session even when the route fails, and uncommitted work is rolled back rather than silently committed. The route performs its commit explicitly before returning success. A session should not escape into background work that might execute after request cleanup; that work should open its own session.

M07 · Would replacing every def with async def make this API faster?

Model answer. No. The database session and hashing work are synchronous. Calling them directly inside async handlers could block the event loop. The current synchronous routes run through FastAPI's thread pool. An async design needs an awaitable driver and session throughout the relevant path, and it should be motivated by measured waiting/concurrency needs rather than syntax alone.

M08 · Why reject true, 12.5, and "1299" as prices?

Model answer. The contract deliberately accepts integer minor units, so booleans, fractional values, and string coercion are inappropriate. Python's bool is closely related to int, so an apparently numeric type declaration alone may not express that strict policy. ProductWrite uses a strict integer with lower and upper bounds. A database check also protects the stored numeric range from direct writers.

M09 · How would PATCH differ from the existing PUT?

Model answer. The existing PUT requires all writable fields. A PATCH extension would define which fields can be omitted and use only explicitly supplied values, often through `model_dump(exclude_unset=True)`. It must distinguish omission from an explicit null and keep ownership fields unwritable. I would use a dedicated schema and tests rather than make every PUT field optional and silently change the contract.

M10 · When does offset pagination become a poor fit?

Model answer. Large offsets can require the database to skip substantial work, and insertions/deletions can shift rows between successive pages. The current bounded, ordered listing is reasonable for a small example. For a large changing dataset, I would consider a cursor based on a stable unique ordering, document how it behaves under concurrent writes, and inspect query plans and indexes.

M11 · How does the ORM help prevent SQL injection?

Model answer. Expressions such as a typed equality comparison keep user values as bound parameters rather than concatenated SQL text. This is different from constructing a raw SQL string with an f-string. The ORM does not make every possible dynamic query safe: user-controlled column names or sort directions need an explicit allowlist. I would review any raw SQL and dynamic identifier handling separately.

M12 · Why verify issuer and audience as well as the signature?

Model answer. A valid signature proves that the token was signed by a trusted key, but it may have been issued for another service or purpose. Issuer and audience narrow that trust to the configured source and recipient. Required expiration and subject claims also need explicit presence and validation. The tests reject incorrect audiences, incorrect issuers, and omitted required claims.

26 · Mock interview: Hard

These questions focus on gaps a happy-path demo can miss. A strong answer identifies the invariant, the failure mechanism, and a test or design response.

H01 · A token has a valid signature. Why might the request still fail?

Model answer. The token may be expired, issued in the future, intended for another audience, from an unexpected issuer, missing required claims, or carrying an invalid subject. Even a fully valid token can refer to a deleted account. After authentication, the requested operation may still violate ownership. I would separate cryptographic/claim checks, account resolution, and authorization so each failure has a clear boundary and test.

H02 · Does clearing a token from the browser implement immediate logout?

Model answer. It removes one copy but does not invalidate a copied or stolen token. In this access-token-only design, such a token remains usable until expiration unless the account is removed or the signing key changes. Immediate per-token revocation needs server-side state such as a denylist or a session/token version, with clear propagation and expiry behavior. That changes the simplicity and availability tradeoffs of stateless verification.

H03 · Can two valid owner updates overwrite each other?

Model answer. Yes. Both requests can read the same old product and commit different replacements; the later commit can overwrite earlier work. An owner check does not provide concurrency control. Depending on requirements, I would use optimistic versioning with a checked version/precondition, an atomic conditional update, or appropriate locking. I would test concurrent behavior on the deployment database rather than infer it from sequential SQLite tests.

H04 · What is an N+1 query problem, and could this API develop one?

Model answer. A list query returns N rows, then serializing or accessing a relationship performs another query for each row. The result can be slow despite individually fast queries. The current product response returns simple attributes and avoids traversing the seller relationship. If nested seller data is added, I would measure query counts and use an appropriate eager-loading strategy rather than permit implicit per-row loads.

H05 · Is a dynamic ORDER BY safe just because the ORM is used?

Model answer. Not automatically. Bound parameters represent values; identifiers such as column names are a different problem. I would map a small set of public sort names to known ORM attributes and explicitly allowed directions. I would reject unknown names and keep a stable tie-breaker. Building raw ORDER BY text from a query string would undermine the parameterized-value protections used elsewhere.

H06 · Why use separate sessions for HTTP requests in tests?

Model answer. A shared session can reveal uncommitted objects and hide missing commits. Separate request sessions make a later request observe what was actually persisted. The in-memory database is still shared intentionally through StaticPool, because TestClient uses worker threads. The fixture overrides only the session dependency, so routing, validation, token handling, and ORM behavior still run together.

H07 · What does 100% test coverage fail to prove?

Model answer. It proves that the measured statements executed, not that all branches, input combinations, invariants, concurrency conditions, or deployment failures were asserted. A test can execute an owner check without verifying unchanged state after denial. The suite should be evaluated by defect detection and meaningful contracts. Real networking, load, dependency outages, security review, and the chosen production database require additional evidence.

H08 · How can password hashing costs be upgraded safely?

Model answer. Store the algorithm parameters with each encoded hash. After a successful login, verify_and_update can produce a replacement using the current settings, which must then be persisted. Do not try to recover a password from its old hash. Measure the resource cost before raising parameters. The repository test creates an account with cheaper Argon2 settings, signs in, and verifies that the stored hash was upgraded and no longer needs replacement.

H09 · Why is committing a product and sending an email not one atomic operation?

Model answer. A relational transaction does not control an external email service. Sending first can notify about a rolled-back change; committing first can leave a committed change without a notification if sending fails. For a durable requirement, I would write an outbox event in the same database transaction and deliver it with a worker. Delivery may repeat, so the worker or receiver needs idempotency and an observable retry policy.

H10 · How can a validation error leak a password even when responses omit hashes?

Model answer. A default validation error can contain the rejected input value. A short or invalid password might therefore be echoed in an error body and captured by logs or tooling. The custom handler retains only location, message, and type, excluding input and context. I would also review request logging and client telemetry; successful response filtering covers only one route by which secrets can escape.

H11 · What changes when the API runs in four worker processes?

Model answer. Each process has its own memory, cached settings, dummy hash, and connection pool. They do not share an in-memory rate limiter, cache, or revocation dictionary. Total database connections can multiply, and SQLite write contention does not disappear. Every worker must receive compatible signing configuration. I would size workers and pools against workload and database capacity, and put shared state in a suitable external system when required.

H12 · Can the SQLite test suite certify a PostgreSQL deployment?

Model answer. No. The API-level contracts are useful across backends, but type behavior, constraint handling, transaction isolation, locking, query plans, and driver behavior can differ. I would install the selected synchronous driver, run migrations on a disposable PostgreSQL instance, execute the relevant integration suite there, and add concurrency tests for the chosen invariants. The database URL alone is not the entire migration plan.

27 · Mock interview: Architect

Architect questions have several defensible answers. State assumptions, choose the important invariants, and explain the operational consequences of the design.

A01 · How would you scale this API from a learning project to a high-traffic service?

Model answer. I would first define traffic shape, write/read ratio, latency goals, availability targets, and recovery requirements. Then I would measure query latency, pool waits, hashing cost, and error patterns. Likely early changes are a managed database, appropriate indexes, ingress rate limits, and supervised stateless API replicas. I would add caching or async I/O for a measured bottleneck, preserving owner checks and transaction semantics. Load tests should include the actual database and realistic authentication behavior, not just an unauthenticated health endpoint.

A02 · How would you introduce enterprise SSO without expanding local auth into an identity platform?

Model answer. I would use an OIDC provider and Authorization Code with PKCE, define trusted issuer/audience and key discovery policies, and map the external stable subject to an internal seller account. The API would keep its own resource ownership and domain permissions. Account linking, tenant membership, deprovisioning, and token lifetimes need explicit policies. I would migrate incrementally and retain clear tests for identity resolution and authorization instead of treating a provider login as permission to every resource.

A03 · How would you add organizations and tenant isolation?

Model answer. I would model organization membership and attach tenant ownership to resources. Tenant context must be resolved from trusted identity/membership information, then enforced in every relevant read and write query. Composite keys/constraints, row-level security where appropriate, and tests with overlapping identifiers can strengthen the boundary. I would review background jobs, caches, exports, and administrative paths as well as HTTP handlers. A caller-supplied tenant_id or a UI filter is insufficient isolation.

A04 · How would you rotate signing keys across services safely?

Model answer. I would define a signing/verification trust boundary, token lifetime, overlap window, and emergency revocation policy. With asymmetric signing, issuers keep private keys while verifiers receive trusted public keys, commonly through a controlled JWKS mechanism. A key ID selects among trusted configured keys; it must not enable arbitrary remote key retrieval. During rotation, new tokens use the new key while old tokens remain verifiable for the allowed overlap. Cache behavior, outages, clock handling, and auditability need tests and operating procedures.

A05 · How would you change the schema during a zero-downtime deployment?

Model answer. I would use an expand-and-contract sequence: add backward-compatible structures, deploy code that tolerates old and new states, backfill in bounded observable steps, switch reads/writes, and remove obsolete structures only after older workers are gone. I would evaluate locks and database-specific migration behavior on realistic data. A rollback of application code must remain compatible with the intermediate schema. Backup/restore and forward-repair plans are more useful than assuming every downgrade is safe.

A06 · How would you make product-change events reliable?

Model answer. I would persist the product change and an outbox record in one database transaction. A worker delivers records with retries, identifiers, and observability. Because delivery can repeat or fail after the receiver acts, consumers need idempotency or an inbox/deduplication boundary. Ordering requirements should be explicit, often per product rather than globally. I would avoid claiming end-to-end exactly-once delivery from a simple queue configuration and test crash points around commit, delivery, and acknowledgment.

A07 · How would you protect login and registration from abuse?

Model answer. I would put shared rate and request-size limits at the ingress before expensive hashing, combine relevant account and network signals, and monitor outcomes without logging credentials. Limits must work across replicas and avoid easy denial of service against legitimate accounts. Depending on risk, progressive challenges, MFA, breached-password checks, or an identity provider may be appropriate. Error responses should remain generic, and the system needs recovery and support policies rather than permanent lockouts that attackers can trigger cheaply.

A08 · Where should a browser client store its authentication state?

Model answer. I would choose after defining the frontend/backend topology and threat model. Script-readable tokens can be stolen by XSS. HttpOnly cookies reduce direct script access but require correct Secure/SameSite settings and a CSRF strategy for state-changing requests. Token lifetime, refresh rotation, logout, cross-origin behavior, and device/session management must be designed together. CORS is not a substitute for those controls. I would document the chosen tradeoff and test browser behavior rather than present one storage location as universally safe.

A09 · What would a credible availability and recovery plan include?

Model answer. I would define service-level objectives plus recovery point and recovery time objectives. The design needs persistent storage, verified backups, restore drills, supervised processes, dependency-aware readiness, and alerts tied to user-visible symptoms. Database failure should not trigger uncontrolled restart loops. I would plan how migrations, secret rotation, and regional or provider failures affect recovery. A health endpoint and automated backups are useful components, but only a tested restore demonstrates that the recovery path works.

A10 · How would you extend this into inventory or purchase processing?

Model answer. I would identify invariants such as non-negative stock, accepted order uniqueness, and permitted state transitions before adding endpoints. Stock changes may require atomic conditional writes or locking, and retried purchase requests need idempotency. Money would need explicit currency and accounting rules. Payment-provider calls require durable workflow and reconciliation rather than a single long database transaction. These are new domain requirements; I would keep them out of the simple reference until the invariant and failure model are defined.

A11 · What would you instrument before a public launch?

Model answer. I would capture request rate, error rate, latency distributions, database/pool signals, and authentication-failure patterns, with request/trace identifiers that connect events. Logs would exclude passwords, tokens, and sensitive bodies. I would define alert thresholds and ownership, exercise failure cases, and track release/migration context. Measurements should answer whether users can register, authenticate, and perform permitted CRUD operations, not merely whether a process is alive.

A12 · When would you introduce services, repositories, or microservices?

Model answer. I would start with cohesive modules and explicit transactions in one deployable service. A service layer becomes useful when business operations are reused or coordinate several rules; a repository helps when query behavior or persistence boundaries need isolation. A separate service is justified by a real ownership, scaling, deployment, or trust boundary and brings network failure and data-consistency costs. I would prefer an understandable modular application over distribution that only turns local calls into unreliable remote ones.

28 · References and further study

This guide's concrete examples come from the accompanying project. The references below support the framework behavior and standards discussed in the text. Consult the documentation for the installed versions when extending the application.

- FastAPI: concurrency and async/await — handler execution models and blocking work.
- FastAPI: request bodies — typed body contracts and validation.
- FastAPI: response models — serialization and public output filtering.
- FastAPI: dependencies — composing reusable providers.
- FastAPI: handling errors — HTTP exceptions and custom handlers.
- FastAPI: security and JWT — bearer authentication integration.
- FastAPI: deployment concepts — process, memory, replication, and operating concerns.
- Pydantic: settings — typed environment configuration.
- SQLAlchemy: sessions — units of work and transaction behavior.
- SQLAlchemy: SQLite — foreign keys, threading, and transactions.
- Alembic: tutorial — versioned schema transitions.
- PyJWT: usage — algorithms and registered claims.
- pwdlib: guide — password verification and gradual upgrades.
- pytest: fixtures — isolated setup and cleanup.
- Starlette: TestClient — in-process HTTP testing.
- OAuth security best current practice, RFC 9700 — password-grant limitations and modern security guidance.

Continue learning deliberately

Rebuild one feature without copying, write a failure-case test first, then explain its trust and transaction boundaries. Good revision is not recalling a decorator name; it is predicting what happens when input is invalid, credentials are stale, two writes race, or a dependency fails.